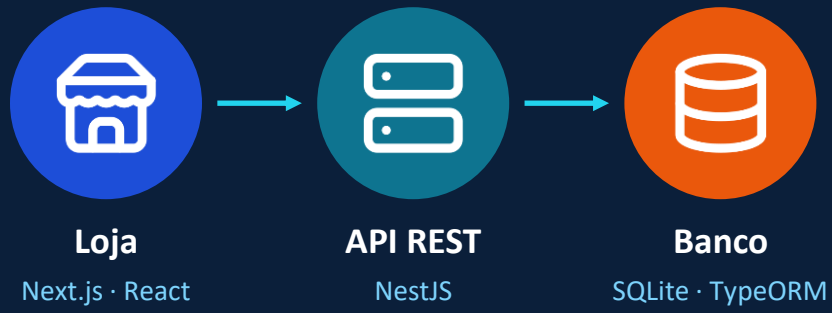
PARTE 2 · BACK-END

TecShop —
Desenvolvimento do Back-end

Parte 2 — Projeto e Desenvolvimento de Sistemas Web

Universidade	Universidade Tecnológica Federal do Paraná (UTFPR)
Câmpus	Santa Helena
Curso	Bacharelado em Ciência da Computação
Disciplina	Projeto e Desenvolvimento de Sistemas Web
Professor	Prof. B.Sc. Wellington de Souza Ferreira



INTEGRANTES

Aluno	R.A.
Gustavo Galafaci Conte	2576635
Igor Carvalho Marchi	2677229
Jefferson korte junior	2651114

CONTEXTO · PROBLEMA

Da aplicação visual à persistência real

Etapa 1 · só front-end

- Next.js 16 + React 19: loja e painel prontos
- Produtos copiados uma vez da Fake Store API
- Usuários, pedidos e promoções no localStorage
- Tipo do usuário num cookie editável

Etapa 2 · back-end próprio

- API REST em NestJS 11, organizada em camadas
- Dados no SQLite, mapeados com TypeORM
- Senha com bcrypt e acesso com JWT
- Preço, estoque e permissão decididos no servidor

O que precisava ser resolvido

- Dados presos a um navegador
- Nenhuma base central
- Painel admin sem servidor
- Senhas em texto puro
- Permissão burlável pelo F12

META Uma API própria que atende a loja, persiste os dados e protege as operações do sistema.

Como a TecShop passou a funcionar?



O controller cuida do caso feliz; a esteira cuida de todo o resto.

A comunicação entre front-end e back-end

Método	Endpoint	Finalidade	Acesso
GET	`/produtos`	Listar com busca, filtros e página	Público
GET	`/produtos/:id`	Consultar um produto	Público
POST	`/produtos`	Cadastrar produto	ADMIN
PATCH	`/produtos/:id`	Alterar só o que veio	ADMIN
DELETE	`/produtos/:id`	Excluir (soft delete)	ADMIN
POST	`/auth/login`	Entrar e receber o JWT	Público
POST	`/pedidos`	Finalizar a compra	Logado
GET	`/pedidos/meus`	Histórico do cliente	Logado

37 **endpoints em 11 recursos**
contados no documento OpenAPI da API

Três jeitos de mandar dados

@Param `/produtos/42`
qual recurso?

@Query `/produtos?busca=fone&pagina=2`
como quero a lista?

@Body `{ "produtoId": 1, "quantidade": 2 }`
quais dados salvar? (item do pedido)

A URL diz o quê, o método diz a ação, o status diz como foi.

200	201	204	400	401	403	404	409	422
consulta ok	criado	sem corpo	dado inválido	sem login	sem permissão	não existe	conflito	regra violada

Controller traduz, Service decide, Repository guarda

Exemplo real: o caminho de um POST /pedidos quando a cliente finaliza a compra

1 Checkout

envia só produtoId e quantidade, mais os dados de entrega

→

2 Controller

uma linha: chama o service com o DTO e o usuário do token

→

3 Service · decide

- abre a transação
- produto existe e está ativo? (404)
- faltou estoque? 409 SEM_ESTOQUE
- preço e total vêm do banco
- dono do pedido vem do token

→

4 Repository

grava pedido e itens (cascade) pelo manager da transação

→

5 Resposta

201 { sucesso: true, dados } e a tela mostra "Pedido #1 realizado!"

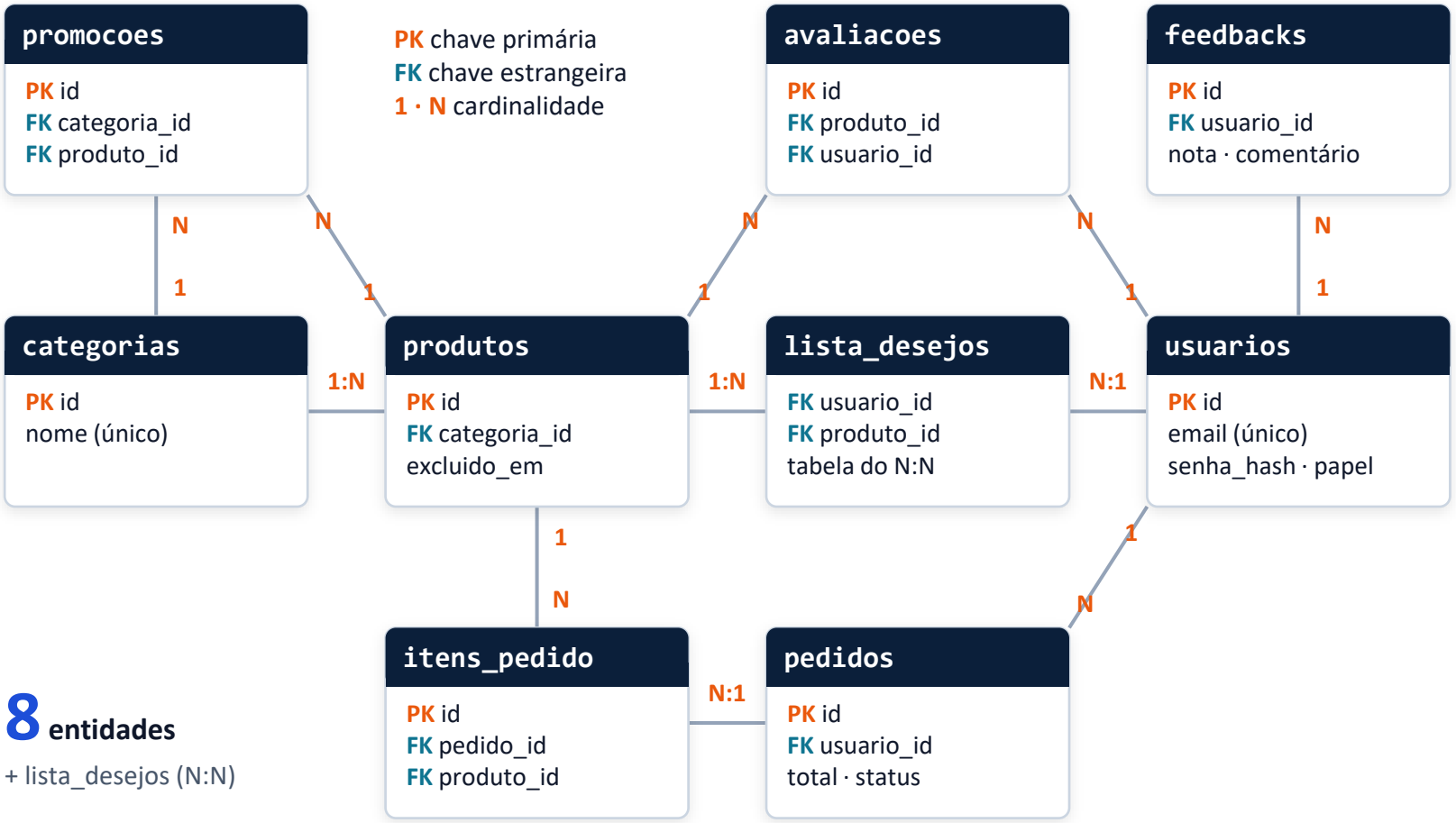
pedidos.controller.ts · o controller só traduz

```
@Post()
@AcessoAutenticado()
criar(
  @Body() dto: CriarPedidoDto,
  @UsuarioAtual() usuario: UsuarioToken,
) {
  return this.pedidosService.criar(dto, usuario);
}
```

produtos.repository.ts · o repository guarda

```
// a condição está no próprio UPDATE: duas compras
// ao mesmo tempo não levam a mesma última unidade
.update(Produto)
.set({ estoque: () => 'estoque - :quantidade' })
.where('id = :id AND estoque >= :quantidade', {
  id,
  quantidade,
})
```

Onde ficam os dados da TecShop?



8 entidades

\+ lista_desejos (N:N)

No 1:N, a FK fica no lado N. O N:N vira uma tabela no meio.

SQLite em arquivo
dados/tecshop.sqlite: reiniciou a API, os dados continuam

TypeORM
@Entity vira tabela; o repository usa await

synchronize só em dev
em produção, a migration cria as tabelas

Seed: npm run seed
admin, 4 categorias e 20 produtos, sem duplicar

Pedido com retrato
o item guarda nome e preço da hora da compra

Como evitamos dados inválidos e erros confusos?

Requisição válida `GET /produtos/1`

Middleware
anota o início

→

ValidationPipe
"1" vira 1

→

Controller · Service
busca o produto

→

Interceptor
embrulha o sucesso

```
{
  "sucesso": true,
  "dados": {
    "id": 1,
    "nome": "Fjallraven - Foldsack No. 1…",
    "preco": 109.95
  }
}
```

Requisição inválida `POST /produtos` `{ "preco": -5, … }`

Middleware
anota o início

→

ValidationPipe
barra com 400

→

Controller · Service
nem é chamado

→

Exception Filter
monta o erro

```
{
  "sucesso": false,
  "erro": {
    "status": 400,
    "mensagem": "Dados inválidos.",
    "detalhes": ["preco deve ser maior que zero"]
  }
}
```

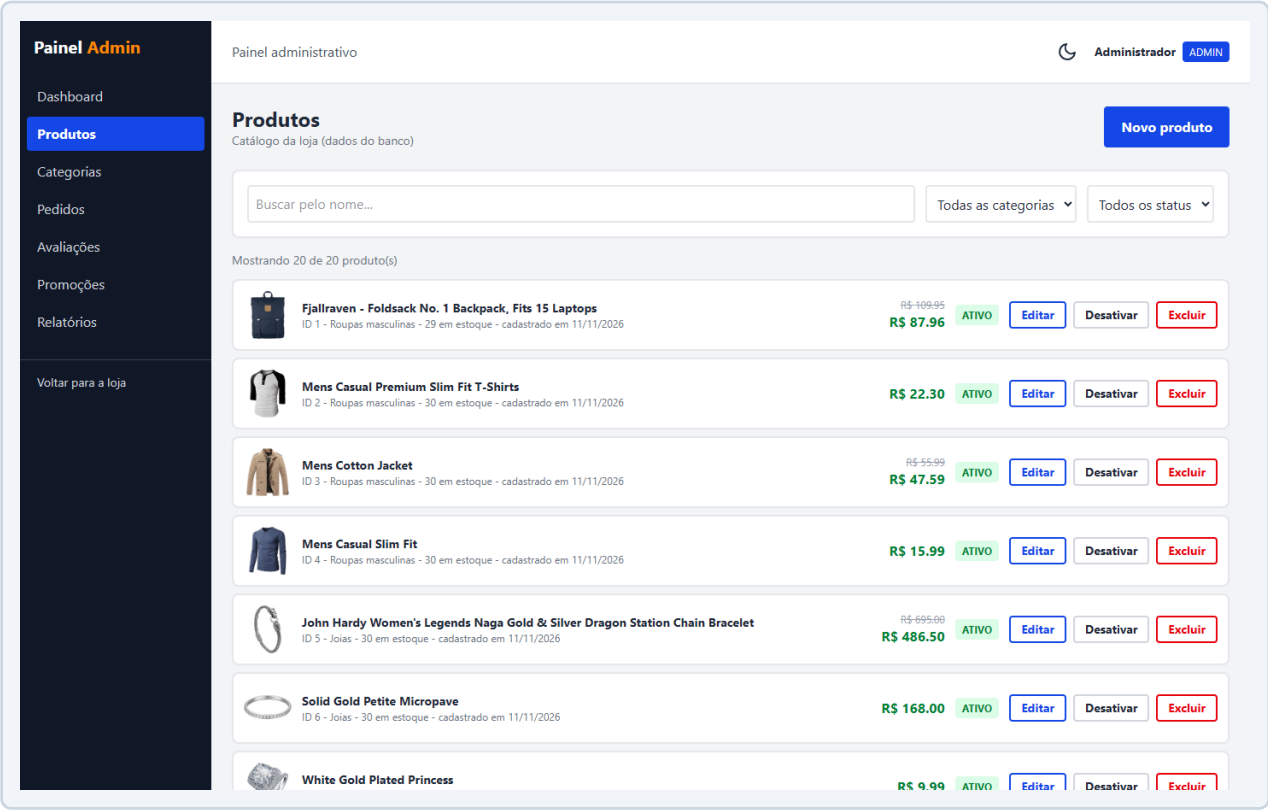
```
[HTTP] POST /pedidos 201 - 19ms
[HTTP] GET /produtos/999 404 - 3ms
[HTTP] POST /produtos 403 - 2ms
[HTTP] POST /pedidos 409 - 6ms
```

Log real do middleware: método, rota, status e tempo. Corpo, senha e token não entram.

SemEstoqueException
409 com código SEM_ESTOQUE

Erro inesperado
500 genérico; detalhe só no log

Criar, consultar, atualizar e excluir



Captura real do painel: busca, filtros e paginação são feitos pela API, no banco.

Validar na entrada, conferir antes de mudar, salvar tudo ou nada.

Operação	Endpoint	TypeORM	Status
Create	`POST /produtos`	`create + save`	201
Read	`GET /produtos?pagina=1`	`findAndCount`	200
Read	`GET /produtos/:id`	`findOne`	200 · 404
Update	`PATCH /produtos/:id`	`busca + save`	200 · 404
Delete	`DELETE /produtos/:id`	`softDelete`	204 · 404

- **Lista paginada no banco:** no máximo 50 por página, com busca e filtros
- **PATCH com PartialType:** só o campo que veio muda; o resto fica igual
- **Conferir antes de mudar:** se não existe, 404; produto excluído some da loja
- **Pedido em transação:** pedido, itens e estoque, tudo ou nada

Quem está acessando e o que pode fazer?

1 · Cadastro e login (uma vez)

Cadastro
o RegistroDto não tem papel: sempre CLIENTE

→

bcrypt
hash com sal, custo 10; senha_hash com select: false

→

Login
compara o hash; erro genérico (401)

→

JWT de 1 hora
{ sub, email, papel }, guardado num cookie httpOnly

2 · Cada requisição depois

Bearer token
o Next envia o JWT no cabeçalho Authorization

→

AuthGuard
quem é você? sem token válido: 401

→

PapeisGuard
o que você pode? cliente em rota ADMIN: 403

→

Service
este recurso é seu? pedido de outro: 404

Perfil	O que pode fazer
CLIENTE	comprar, ver os próprios pedidos, lista de desejos, avaliações e feedback
ADMIN	tudo do cliente + produtos, categorias, promoções, pedidos, usuários e relatórios

O dono do pedido vem do token, nunca do corpo da requisição.

Respostas reais da nossa API

- **400** POST /auth/registro com papel ADMIN: property papel should not exist
- **401** GET /pedidos/meus sem token
- **403** POST /produtos como CLIENTE: Acesso permitido apenas para: ADMIN.
- **404** GET /pedidos/1 de outra cliente: Pedido 1 não encontrado.

Front-end e back-end trabalhando juntos

	Antes (etapa 1)	Depois (etapa 2)
Produtos	Fake Store + localStorage	**GET /produtos, do SQLite**
Login	conferido no navegador	**POST /auth/login + JWT**
Pedidos	localStorage	**POST /pedidos em transação**
Painel	dados de cada navegador	**rotas [ADMIN] da API**
Permissão	cookie editável	**AuthGuard + PapeisGuard**

Por onde passa uma chamada do front

Tela
React, sem fetch na página

→

lib/api.js
res.ok, envelope e Zod

→

/api (Next)
cookie httpOnly vira Bearer

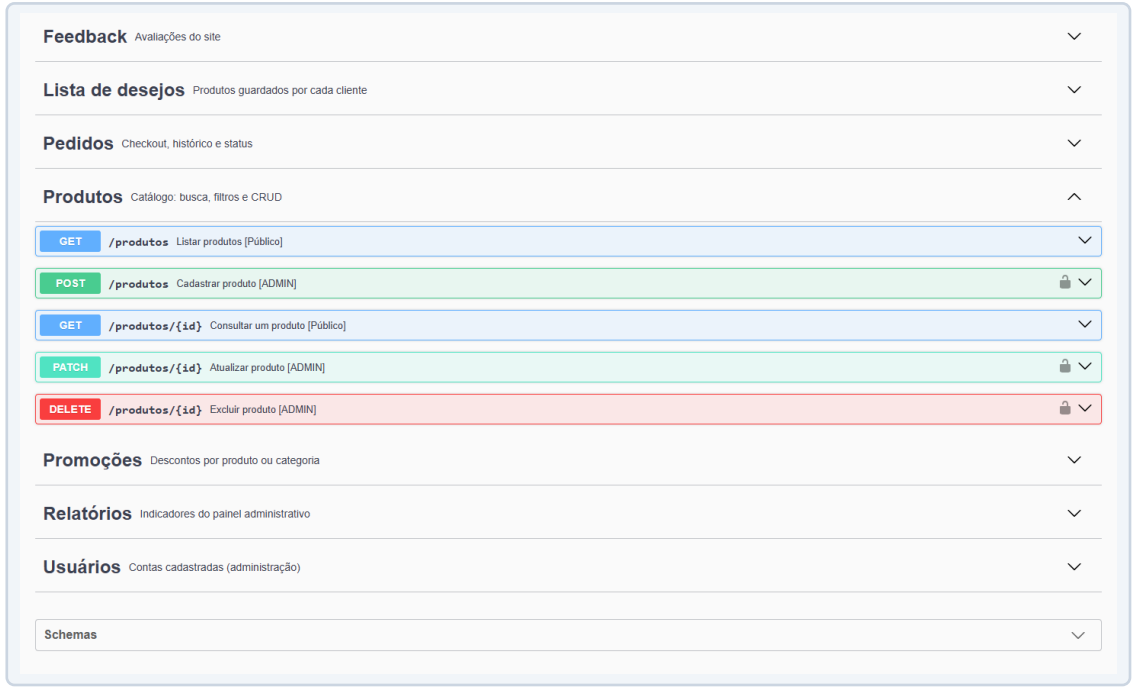
→

API NestJS
guards, DTO, service

O proxy.js confere a assinatura do JWT antes de abrir as páginas privadas.

Swagger UI em /docs

Recurso complementar



- Gerado a partir dos controllers e DTOs: 37 endpoints em 11 grupos
- Authorize com o JWT e Try it out em cada rota
- Cadeado nas rotas protegidas; [Público] e [ADMIN] no resumo

RESULTADOS

O que conseguimos entregar?

Requisito	Situação	Como está no código
API REST	**Implementado**	37 endpoints em 11 recursos
Arquitetura em camadas	**Implementado**	controller → service → repository
SQLite e TypeORM	**Implementado**	8 entidades + N:N; migration
CRUD	**Implementado**	paginação, PATCH, soft delete, transação
Validação e erros	**Implementado**	DTOs, ValidationPipe, envelope único
bcrypt e JWT	**Implementado**	hash custo 10; token de 1 hora
Guards e permissões	**Implementado**	AuthGuard, PapeisGuard, dono do recurso
Integração com o front	**Implementado**	telas na API (usuários só pelo Swagger)
Swagger (complementar)	**Implementado**	/docs com Authorize e Try it out

26/26
testes unitários dos services (Jest)

57/57
testes de integração, com banco temporário

19
cenários da tabela de testes, todos passando

0
erros no build e no lint

Próximos passos
refresh token · promoções ligadas pela data · helmet e limite de tentativas no login · upload de imagens

A TecShop evoluiu de uma aplicação dependente de dados locais para uma arquitetura com API própria, persistência de dados e controle de acesso no servidor.